

An Introduction to Character Device Driver Development

Table of Contents

Introduction	2
Procedure	2
1. PetaLinux Setup and Project Preparation	2
2. Creation of Multiplier Character Device Driver	2
3. BitBake File Modification.....	3
4. User Application Development.....	3
5. Build and Deployment	3
6. Execution on Zybo Board.....	3
Results	4
Conclusion	4
Knowledge Transfer	5
Appendix	6
APPENDIX A (Build Success)	6
APPENDIX B (System Boot)	7
APPENDIX C (Module Insertion)	7
APPENDIX D (Device Node Creation)	8
APPENDIX E (Device Operation Results)	8
APPENDIX F (Source Code).....	10

Introduction

The objective of this laboratory exercise was to develop a **character device driver** in an embedded Linux environment on the Zybo Z7-10 board using PetaLinux. This lab builds upon the kernel module developed in the previous lab and extends it to support user-space interaction through a device file. The device driver acts as an interface between user applications and the multiplication hardware peripheral.

Unlike the previous lab where interaction with hardware was performed entirely within kernel space, this lab introduces communication between **user space and kernel space** using standard system calls such as open, read, write, and close. A user application was also developed to test the functionality of the device driver by accessing the device file /dev/multiplier. This exercise provided practical exposure to character device driver implementation, system call handling, and safe data transfer between user space and kernel space

Procedure

1. PetaLinux Setup and Project Preparation

1. A PetaLinux installation directory was created under /tmp for the user workspace:
`cd /tmp`
`mkdir petalinux_nithingowtham25`
2. The PetaLinux installer was copied from the shared lab directory:
`cd petalinux_nithingowtham25`
`cp /mnt/lab_files/ECEN449/petalinux-v2023.1-05012318-installer.run .`
3. A directory was created for installation and the installer was executed:
`mkdir petalinux_install`
`chmod 755 ./petalinux-v2023.1-05012318-installer.run`
`./petalinux-v2023.1-05012318-installer.run --dir petalinux_install --platform "arm"`
4. After installation, environment variables were loaded:
`source petalinux_install/settings.sh`
5. A new PetaLinux project was created using the Zynq template:
`petalinux-create -t project --template zynq -n linux_boot_nithingowtham25 --tmpdir /tmp/petalinux_nithingowtham25/tmp_dir`
6. The hardware configuration was updated using the exported Vivado design:
`cd linux_boot_nithingowtham25`
`petalinux-config --get-hw-description=/home/grads/n/nithingowtham25/Spring2026/ECEN749/lab4_new/linux_boot_wrapper.xsa --silentconfig`

2. Creation of Multiplier Character Device Driver

7. A kernel module named multiplier was created:
`petalinux-create -t modules --name multiplier --enable`

8. The multiplier.c file was implemented based on the character device framework.
9. The following key functionalities were added:
 - a. Dynamic registration of character device (major number assigned by kernel)
 - b. Initialization routine with memory mapping using ioremap()
 - c. Exit routine with device unregistration and iounmap()
10. File operations implemented:
 - a. open() → prints message when device is opened
 - b. close() → prints message when device is closed
 - c. read() → transfers data from hardware to user buffer using put_user()
 - d. write() → transfers data from user buffer to hardware using get_user()
11. Address constraints enforced:
 - a. Read operations: bytes 0–11
 - b. Write operations: bytes 0–7

The successful compilation of the kernel module is shown in [Appendix A](#).

3. BitBake File Modification

10. The multiplier.bb file was updated to include required header files such as multiplier.h, xparamters.h, xparameters_ps.h.

4. User Application Development

11. A user-space application devtest.c was created outside the PetaLinux project.
12. The following system calls were used:
 - open() → open /dev/multiplier
 - write() → send operands to device
 - read() → retrieve multiplication results
 - close() → release device
13. Nested loops (0–16) were implemented to test multiple multiplication cases.
14. Results were validated by comparing hardware output with software-computed values.

5. Build and Deployment

15. The project was built using:
petalinux-build
16. The generated multiplier.ko module was located in the build directory.
17. The following files were copied to the SD card: BOOT.BIN , image.ub, boot.scr, multiplier.ko, devtest

6. Execution on Zybo Board

18. The system was booted using the SD card (shown in Appendix B).
19. The SD card was mounted:
mount /dev/mmcbk0p1 /mnt/
20. The kernel module was inserted:
insmod /mnt/multiplier.ko

The module insertion output is shown in [Appendix C](#).

21. The device node was created using mknod as instructed by kernel output (shown in [Appendix D](#)).

22. The user application was executed:

```
cd /mnt/  
./devtest
```

23. The functionality was verified through correct multiplication outputs (shown in [Appendix E](#)).

(The complete working of the system was demonstrated to the TA)

Results

The embedded Linux system successfully booted on the Zybo Z7-10 board, as shown in [Appendix B](#), and the character device driver was compiled and loaded correctly, as shown in [Appendix A](#) and [Appendix C](#). The multiplier module registered successfully, and the kernel printed the dynamically assigned major number.

The device node `/dev/multiplier` was created successfully using the `mknod` command, as shown in [Appendix D](#). The user application was able to open the device, write operands, read results, and close the device without errors.

The multiplication peripheral returned correct results for all tested input combinations (0–16). The execution flow is demonstrated in [Appendix E](#), where:

- The device open operation is shown in `result_1_opened.png`
- Correct multiplication functionality is verified in `result_2_proper_working.png`
- The device close operation is shown in `result_3_closed.png`

The following key observations were made:

- Device open and close operations were correctly logged
- Data transfer between user space and kernel space was successful
- Hardware results matched expected software results
- The system demonstrated stable and correct operation

The successful execution of both modules was demonstrated to the TA.

Conclusion

This laboratory exercise successfully demonstrated the implementation of a **character device driver** in an embedded Linux system. The development process, including module compilation ([Appendix A](#)), system boot ([Appendix B](#)), module insertion ([Appendix C](#)), and device node creation ([Appendix D](#)), confirmed correct integration between software and hardware.

The execution of the user application and validation of multiplication results ([Appendix E](#)) highlighted the effectiveness of user-space and kernel-space interaction through device files.

The implementation of system calls such as `open`, `read`, `write`, and `close` reinforced understanding of file-based device interaction, while the use of **`get_user()`** and **`put_user()`** ensured safe data transfer.

Overall, this lab provided a complete view of embedded system design involving hardware, kernel modules, and user applications, forming a strong foundation for advanced device driver development.

Knowledge Transfer

a) Do you think the overall wall-clock time required to perform a multiplication will be faster in this lab compared to the original Lab 3 implementation? Why?

No, the overall wall-clock time in this lab is slower compared to Lab 3. In Lab 3, the multiplication operation was performed using direct hardware access in a bare-metal environment, resulting in minimal overhead. In contrast, this lab involves multiple layers, including user space, system calls, kernel space, and the device driver before reaching the hardware. These additional layers introduce context switching and system call overhead, increasing execution time.

b) Contrast the approach in this lab with that of Lab 3. What are the benefits and costs associated with each approach?

Benefits of the approach in this lab (Character Device Driver / Embedded Linux):

- A higher level of abstraction is provided to the user. The hardware details such as register access and memory mapping are hidden behind the device driver interface. Users can interact with the device using standard system calls (open, read, write, close) without needing knowledge of the underlying hardware.
- The design is more modular and scalable. The same user-space application can work with different hardware implementations as long as the driver interface remains consistent. This improves portability and maintainability.
- The operating system provides additional features such as protection, scheduling, and resource management, allowing multiple applications to safely access the device.

Costs of the approach in this lab:

- Increased execution time due to additional overhead from system calls, context switching, and kernel mediation between user space and hardware.
- Higher implementation complexity, as device driver development requires understanding of kernel programming, synchronization, and hardware interaction.
- Reduced visibility and control over low-level hardware behavior, which can make debugging and optimization more difficult.

Benefits of the approach in Lab 3 (Bare-metal / Direct Hardware Access):

- Lower latency and faster execution since the processor directly interacts with the hardware without involving an operating system or driver layer.
- More efficient use of system resources (CPU cycles and memory), as there is minimal software overhead.
- Greater control and visibility of hardware operations, making it easier to debug and optimize performance-critical sections.

Costs of the approach in Lab 3:

- Requires detailed knowledge of the hardware design, including register maps and FPGA configuration, making development more complex at the hardware level.
- Limited scalability and flexibility, as the implementation is tightly coupled to a specific hardware design and cannot be easily reused or extended.
- Lack of abstraction and protection, which can lead to unsafe operations and reduced robustness in larger systems.

- c) **Explain why it is important that the device registration is the last thing that is done in the initialization routine of a device driver. Likewise, explain why un-registering a device must happen first in the exit routine of a device driver.**

The device registration is intentionally performed as the final step in the initialization routine to ensure that the driver is fully configured before it becomes accessible to the kernel and user-space applications. Once the device is registered, the kernel can immediately invoke the driver's file operations such as open, read, and write. If registration occurs before completing critical initialization steps such as memory allocation, hardware mapping (via `ioremap()`), or internal state setup, the driver may be accessed in an incomplete or inconsistent state. This can result in race conditions, invalid memory access, or undefined behavior. Therefore, registration must occur only after the driver is fully initialized and ready to handle requests safely.

Conversely, device unregistration is performed as the first step in the exit routine to immediately revoke access to the driver. By unregistering early, the kernel is prevented from issuing any new operations to the device while cleanup is ongoing. This ensures that no file operations are executed while resources such as memory mappings or buffers are being released. This ordering guarantees proper synchronization, prevents use-after-free errors, and ensures a safe and deterministic shutdown of the device driver.

Appendix

APPENDIX A (Build Success)

```
nithingowtham25@zach-333-em4-07:/tmp/petalinux_nithingowtham25/linux_boot_nit...
nithingowtham25@zach-333-e... x nithingowtham25@zach-333-e... x nithingowtham25@zach-333-em... x
in SIGGEN_LOCKEDSIGS_t-cortexa9t2hf-neon
The cracklib-native:do_populate_lic sig is computed to be fb8c48d2286f834773a054c9b6d1455ee3e16b9815
9b23a698ea99427a14c442, but the sig is locked to 059f83c7dcd22dda2a903132eead0fa4683ba47834aaf85c518
aaadb3cda2fca in SIGGEN_LOCKEDSIGS_t-x86-64
The cracklib:do_populate_lic sig is computed to be 968af2fd6d5dafd0adde0c99d65a5b431f0dbe2db71b85b6e
98df3bddc95462e, but the sig is locked to 6359632f75f1183510a4567460fc9ff1a97ce4ad1d9b9a2052da3614dc
bd311c in SIGGEN_LOCKEDSIGS_t-cortexa9t2hf-neon
Removing 2 stale sstate objects for arch zynq_generic_7z010: 100% |#####| Time: 0:00:00
NOTE: Executing Tasks
Setscene tasks: 1094 of 1564
Setscene tasks: 1564 of 1564
Setscene tasks: 1564 of 1564
Setscene tasks: 1564 of 1564
Setscene tasks: 1564 of 1564
Setscene tasks: 1564 of 1564
Setscene tasks: 1564 of 1564
Setscene tasks: 1564 of 1564
Setscene tasks: 1564 of 1564
Setscene tasks: 1564 of 1564
WARNING: multiplier-1.0-r0 do_package_qa: QA Issue: Recipe LICENSE includes obsolete licenses GPLv2
[obsolete-license]
Setscene tasks: 1564 of 1564
Setscene tasks: 1564 of 1564
Setscene tasks: 1564 of 1564
Setscene tasks: 1564 of 1564
Setscene tasks: 1564 of 1564
Setscene tasks: 1564 of 1564
NOTE: Tasks Summary: Attempted 4169 tasks of which 2895 didn't need to be rerun and all succeeded.
Summary: There were 2 WARNING messages.
INFO: Failed to copy built images to tftp dir: /tftpboot
[INFO] Successfully built project
nithingowtham25@zach-333-em4-07:/tmp/petalinux_nithingowtham25/linux_boot_nithingowtham25$
```

APPENDIX B (System Boot)

```
nithingowtham25@zach-333-em4-07:~/Spring_2026/ECEN749
nithingowtham25@zach-333-e... x nithingowtham25@zach-333-e... x nithingowtham25@zach-333-em... x
INIT: version 3.04 booting
Starting udev
Starting version 251.8+
ext3: Unknown parameter 'umask'
ext2: Unknown parameter 'umask'
ext4: Unknown parameter 'umask'
FAT-fs (mmcblk0p1): Volume was not properly unmounted. Some data may be corrupt. Please run fsck.
hwclock: can't open '/dev/misc/rtc': No such file or directory
Fri Mar  9 12:34:56 UTC 2018
hwclock: can't open '/dev/misc/rtc': No such file or directory
random: crng init done
Configuring packages on first boot....
(This may take several minutes. Please do not power off the machine.)
Running postinst /etc/rpm-postinsts/100-sysvinit-inittab...
update-rc.d: /etc/init.d/run-postinsts exists during rc.d purge (continuing)
Removing any system startup links for run-postinsts ...
/etc/rcS.d/S99run-postinsts
INIT: Entering runlevel: 5
Configuring network interfaces... Cannot find device "eth0"
Starting OpenBSD Secure Shell server: sshd
generating ssh RSA host key...
generating ssh ECDSA host key...
generating ssh ED25519 host key...
done.
Starting rpcbind daemon...done.
starting statd: done
hwclock: can't open '/dev/misc/rtc': No such file or directory
Starting internet superserver: inetd.
NFS daemon support not enabled in kernel
Starting syslogd/klogd: done
Starting tcf-agent: OK

Petalinux 2023.1+release-S05010539 linux_boot_nithingowtham25 ttyPS0

linux_boot_nithingowtham25 login: petalinux
You are required to change your password immediately (administrator enforced).
New password:
Retype new password:
linux_boot_nithingowtham25:~$
```

APPENDIX C (Module Insertion)

```
linux_boot_nithingowtham25:~$ sudo su

We trust you have received the usual lecture from the local System
Administrator. It usually boils down to these three things:

    #1) Respect the privacy of others.
    #2) Think before you type.
    #3) With great power comes great responsibility.

Password:
linux_boot_nithingowtham25:/home/petalinux# mount /dev/mmcblk0p1 /mnt/
/dev/mmcblk0p1: Can't open blockdev
/dev/mmcblk0p1: Can't open blockdev
/dev/mmcblk0p1: Can't open blockdev
linux_boot_nithingowtham25:/home/petalinux# insmod /mnt/multiplier.ko
multiplier: loading out-of-tree module taints kernel.
Mapping virtual address...
Physical address = 0x43c00000
Virtual address = 27ebff73
Registered a device with dynamic Major number of 244
Create a device file for this device with this command:
'mknod /dev/multiplier c 244 0'.
linux_boot_nithingowtham25:/home/petalinux#
```

APPENDIX D (Device Node Creation)

```
linux_boot_nithingowtham25:/home/petalinux# insmod /mnt/multiplier.ko
multiplier: loading out-of-tree module taints kernel.
Mapping virtual address...
Physical address = 0x43c00000
Virtual address = 27ebff73
Registered a device with dynamic Major number of 244
Create a device file for this device with this command:
'mknod /dev/multiplier c 244 0'.
linux_boot_nithingowtham25:/home/petalinux# mknod /dev/multiplier c 244 0
linux_boot_nithingowtham25:/home/petalinux#
```

APPENDIX E (Device Operation Results)

- result_1_opened

```
linux_boot_nithingowtham25:/home/petalinux# mknod /dev/multiplier c 244 0
linux_boot_nithingowtham25:/home/petalinux# cd /mnt/
linux_boot_nithingowtham25:/mnt# ./devtest
The multiplier device is opened successfully!!!
Multiplier: Write requested = 8 bytes, writing = 8 bytes
Multiplier: Wrote byte[0] = 0
Multiplier: Wrote byte[1] = 0
Multiplier: Wrote byte[2] = 0
Multiplier: Wrote byte[3] = 0
Multiplier: Wrote byte[4] = 0
Multiplier: Wrote byte[5] = 0
Multiplier: Wrote byte[6] = 0
Multiplier: Wrote byte[7] = 0
Multiplier: Total bytes written = 8
Multiplier: Read requested = 12 bytes, serving = 12 bytes
Multiplier: Read byte[0] = 0
Multiplier: Read byte[1] = 0
Multiplier: Read byte[2] = 0
Multiplier: Read byte[3] = 0
Multiplier: Read byte[4] = 0
Multiplier: Read byte[5] = 0
Multiplier: Read byte[6] = 0
Multiplier: Read byte[7] = 0
Multiplier: Read byte[8] = 0
Multiplier: Read byte[9] = 0
Multiplier: Read byte[10] = 0
Multiplier: Read byte[11] = 0
Multiplier: Total bytes read = 12
0 * 0 = 0 Result Correct!
Multiplier: Write requested = 8 bytes, writing = 8 bytes
Multiplier: Wrote byte[0] = 0
Multiplier: Wrote byte[1] = 0
Multiplier: Wrote byte[2] = 0
Multiplier: Wrote byte[3] = 0
Multiplier: Wrote byte[4] = 1
Multiplier: Wrote byte[5] = 0
Multiplier: Wrote byte[6] = 0
Multiplier: Wrote byte[7] = 0
Multiplier: Total bytes written = 8
Multiplier: Read requested = 12 bytes, serving = 12 bytes
Multiplier: Read byte[0] = 0
Multiplier: Read byte[1] = 0
Multiplier: Read byte[2] = 0
Multiplier: Read byte[3] = 0
Multiplier: Read byte[4] = 1
Multiplier: Read byte[5] = 0
Multiplier: Read byte[6] = 0
```


- result_2_proper_working

```
nithingowtham25@zach-333-em4-07:~/Spring_2026/ECEN749
nithingowtham25@zach-333-e... x nithingowtham25@zach-333-e... x nithingowtham25@zach-333-em... x
Multiplier: Total bytes read = 12
11 * 14 = 154 Result Correct!
Multiplier: Write requested = 8 bytes, writing = 8 bytes
Multiplier: Wrote byte[0] = 11
Multiplier: Wrote byte[1] = 0
Multiplier: Wrote byte[2] = 0
Multiplier: Wrote byte[3] = 0
Multiplier: Wrote byte[4] = 15
Multiplier: Wrote byte[5] = 0
Multiplier: Wrote byte[6] = 0
Multiplier: Wrote byte[7] = 0
Multiplier: Total bytes written = 8
Multiplier: Read requested = 12 bytes, serving = 12 bytes
Multiplier: Read byte[0] = 11
Multiplier: Read byte[1] = 0
Multiplier: Read byte[2] = 0
Multiplier: Read byte[3] = 0
Multiplier: Read byte[4] = 15
Multiplier: Read byte[5] = 0
Multiplier: Read byte[6] = 0
Multiplier: Read byte[7] = 0
Multiplier: Read byte[8] = 165
Multiplier: Read byte[9] = 0
Multiplier: Read byte[10] = 0
Multiplier: Read byte[11] = 0
Multiplier: Total bytes read = 12
11 * 15 = 165 Result Correct!
Multiplier: Write requested = 8 bytes, writing = 8 bytes
Multiplier: Wrote byte[0] = 11
Multiplier: Wrote byte[1] = 0
Multiplier: Wrote byte[2] = 0
Multiplier: Wrote byte[3] = 0
Multiplier: Wrote byte[4] = 16
Multiplier: Wrote byte[5] = 0
Multiplier: Wrote byte[6] = 0
Multiplier: Wrote byte[7] = 0
Multiplier: Total bytes written = 8
Multiplier: Read requested = 12 bytes, serving = 12 bytes
Multiplier: Read byte[0] = 11
Multiplier: Read byte[1] = 0
Multiplier: Read byte[2] = 0
Multiplier: Read byte[3] = 0
Multiplier: Read byte[4] = 16
Multiplier: Read byte[5] = 0
Multiplier: Read byte[6] = 0
Multiplier: Read byte[7] = 0
Multiplier: Read byte[8] = 176
Multiplier: Read byte[9] = 0
Multiplier: Read byte[10] = 0
Multiplier: Read byte[11] = 0
Multiplier: Total bytes read = 12
11 * 16 = 176 Result Correct!
Multiplier: Write requested = 8 bytes, writing = 8 bytes
```

- result_3_closed

```
Multiplier: Write requested = 8 bytes, writing = 8 bytes
Multiplier: Wrote byte[0] = 16
Multiplier: Wrote byte[1] = 0
Multiplier: Wrote byte[2] = 0
Multiplier: Wrote byte[3] = 0
Multiplier: Wrote byte[4] = 16
Multiplier: Wrote byte[5] = 0
Multiplier: Wrote byte[6] = 0
Multiplier: Wrote byte[7] = 0
Multiplier: Total bytes written = 8
Multiplier: Read requested = 12 bytes, serving = 12 bytes
Multiplier: Read byte[0] = 16
Multiplier: Read byte[1] = 0
Multiplier: Read byte[2] = 0
Multiplier: Read byte[3] = 0
Multiplier: Read byte[4] = 16
Multiplier: Read byte[5] = 0
Multiplier: Read byte[6] = 0
Multiplier: Read byte[7] = 0
Multiplier: Read byte[8] = 0
Multiplier: Read byte[9] = 1
Multiplier: Read byte[10] = 0
Multiplier: Read byte[11] = 0
Multiplier: Total bytes read = 12
16 * 16 = 256 Result Correct!
Press 'q' to quit or any key to continue...
q
The multiplier device is closed successfully!!!
linux boot nithingowtham25:/mnt#
```

APPENDIX F (Source Code)

- multiplier.h

```
6-character-device-driver > multiplier > C multiplier.h > init_module(void)
1  /* All of our linux kernel includes. */
2  #include <linux/module.h> /* Needed by all modules */
3  #include <linux/moduleparam.h> /* Needed for module parameters */
4  #include <linux/kernel.h> /* Needed for printk and KERN_* */
5  #include <linux/init.h> /* Need for __init macros */
6  #include <linux/io.h>
7  #include <asm/io.h> /* Needed for IO reads and writes */
8  #include <linux/ioport.h>
9  #include <linux/slab.h>
10 #include <linux/fs.h> /* Provides file ops structure */
11 #include <linux/sched.h> /* Provides access to the "current" process
12 | task structure */
13 #include <asm/uaccess.h> /* Provides utilities to bring user space
14 | data into kernel space. Note, it is
15 | processor arch specific. */
16
17
18 /* Some defines */
19 #define DEVICE_NAME "multiplier"
20 #define BUF_LEN 80
21
22 /* Function prototypes, so we can setup the function pointers for dev
23 | file access correctly. */
24 int init_module(void);
25 void my_exit(void);
26 static int device_open(struct inode *, struct file *);
27 static int device_close(struct inode *, struct file *);
28 static ssize_t device_read(struct file *, char *, size_t, loff_t *);
29 static ssize_t device_write(struct file *, const char *, size_t, loff_t *);
30
31 /*
32 | * Global variables are declared as static, so are global but only
33 | * accessible within the file.
34 | */
35 static int Major; /* Major number assigned to our device driver */
36 static int Device_Open = 0; /* Flag to signify open device */
```

- multiplier.c

6-character-device-driver > multiplier > C multiplier.c > ...

```

11  /* Moved all prototypes and includes into the headerfile */
12  #include "multiplier.h"
13  #include "xparameters.h" /* needed for physical address of multiplier */
14
15  /* from xparameters.h */
16  #define PHY_ADDR XPAR_MULTIPLY_0_S00_AXI_BASEADDR /* physical address of multiplier */
17
18  /* size of physical address range for multiply */
19  #define MEMSIZE (XPAR_MULTIPLY_0_S00_AXI_HIGHADDR - XPAR_MULTIPLY_0_S00_AXI_BASEADDR + 1)
20
21  void *virt_addr; /* virtual address pointing to multiplier */
22
23  /* This structure defines the function pointers to our functions for
24   * opening, closing, reading and writing the device file. There are
25   * lots of other pointers in this structure which we are not using,
26   * see the whole definition in linux/fs.h */
27  static struct file_operations fops = {
28      .read = device_read,
29      .write = device_write,
30      .open = device_open,
31      .release = device_close
32  };

```

```

44  int my_init(void)
45  {
46
47      /* Linux kernel's version of printf */
48      printk(KERN_INFO "Mapping virtual address...\n");
49
50      /* map virtual address to multiplier physical address */
51      /* use ioremap */
52      virt_addr = ioremap(PHY_ADDR, MEMSIZE);
53
54      if (!virt_addr) {
55          printk(KERN_ALERT "ioremap failed!\n");
56          return -ENOMEM;
57      }
58
59      /* print physical and virtual addresses */
60      printk(KERN_INFO "Physical address = %x\n", PHY_ADDR);
61      printk(KERN_INFO "Virtual address = %p\n", virt_addr);
62
63
64      /* This function call registers a device and returns a major number
65       * associated with it. Be wary, the device file could be accessed
66       * as soon as you register it, make sure anything you need (ie
67       * buffers ect) are setup BEFORE you register the device.*/
68      Major = register_chrdev(0, DEVICE_NAME, &fops);
69
70
71      /* Negative values indicate a problem */
72      if (Major < 0) {
73          /* Make sure you release any other resources you've already
74           * grabbed if you get here so you don't leave the kernel in a
75           * broken state. */
76          printk(KERN_ALERT "Registering char device failed with %d\n", Major);
77          return Major;
78      }
79
80      printk(KERN_INFO "Registered a device with dynamic Major number of %d\n", Major);
81
82      printk(KERN_INFO "Create a device file for this device with this command:\nmknod /dev/%s c %d 0'.\n", DEVICE_NAME, Major);

```

```

84      return 0; /* success */
85  }
86
87  /*
88   * This function is called when the module is unloaded, it releases
89   * the device file.
90   */
91  void my_exit(void)
92  {
93      /* Unregister the device */
94      unregister_chrdev(Major, DEVICE_NAME);
95
96      /* Unmap virtual memory */
97      printk(KERN_ALERT "unmapping virtual address space....\n");
98      iounmap((void *)virt_addr);
99
100  }

```

```

107 static int device_open(struct inode *inode, struct file *file)
108 {
109     /*
110      * In these case we are only allowing one process to hold the device
111      * file open at a time. */
112     if (Device_Open) /* Device_Open is my flag for the usage of the device
113                      * file (defined in multiplier.h) */
114         return -EBUSY; /* Failure to open device is given
115                      * back to the userland program. */
116     Device_Open++; /* Keeping the count of the device opens. */
117
118
119     try_module_get(THIS_MODULE); /* increment the module use count
120                                * (make sure this is accurate or you
121                                * won't be able to remove the module
122                                * later. */
123
124     /* Print kernel message */
125     printk(KERN_INFO "The multiplier device is opened successfully!!!\n");
126     return 0;
127 }
128
129 /*
130  * Called when a process closes the device file.
131  */
132 static int device_close(struct inode *inode, struct file *file)
133 {
134     Device_Open--; /* We're now ready for our next caller */
135
136     /*
137      * Decrement the usage count, or else once you opened the file,
138      * you'll never get rid of the module.
139      */
140     module_put(THIS_MODULE);
141
142     /* Print kernel message */
143     printk(KERN_INFO "The multiplier device is closed successfully!!!\n");
144     return 0;
145 }

```

```

151 static ssize_t device_read(struct file *filp, /* see include/linux/fs.h */
152                          char __user *buffer, /* buffer to fill with data */
153                          size_t length, /* length of the buffer */
154                          loff_t * offset)
155 {
156     int i;
157     int bytes_read = 0;
158
159     /*
160      * The user may request any number of bytes, but the multiplier
161      * peripheral only supports addresses from byte 0 to byte 11.
162      * So we compute a safe length that does not exceed 12 bytes.
163      */
164     size_t valid_len = (length <= 12) ? length : 12;
165
166     /* Print how many bytes were requested vs how many will be served */
167     printk(KERN_INFO "Multiplier: Read requested = %zu bytes, serving = %zu bytes\n",
168          length, valid_len);
169
170     /*
171      * Loop through each valid byte in the peripheral address space.
172      * For each byte:
173      * - Read from the hardware using ioread32()
174      * - Copy it to user space using put_user()
175      */
176     for (i = 0; i < valid_len; i++) {
177
178         /* Read one byte from the memory-mapped peripheral */
179         char data = ioread32(virt_addr + i);
180
181         /*
182          * Copy the byte from kernel space to user space buffer.
183          * 'buffer' is a user-space pointer, so put_user is required.
184          */
185         put_user(data, buffer + i);
186
187         /* Increment count of successfully transferred bytes */
188         bytes_read++;

```

```

        printk(KERN_INFO "Multiplier: Read byte[%d] = %d\n", i, data);
    }

    /*
     * Return the number of bytes actually transferred to user space.
     * This satisfies the requirement of the read() system call.
     */
    printk(KERN_INFO "Multiplier: Total bytes read = %d\n", bytes_read);

    return bytes_read;
}

```

```

207 static ssize_t device_write(struct file *file, const char __user * buffer, size_t length, loff_t * offset)
208 {
209     int i;
210     int bytes_written = 0;
211
212     /*
213      * The multiplier peripheral only supports writes to byte
214      * addresses 0 through 7. So we compute a safe length that
215      * does not exceed 8 bytes.
216      */
217     size_t valid_len = (length <= 8) ? length : 8;
218
219     /* Print how many bytes were requested vs how many will be written */
220     printk(KERN_INFO "Multiplier: Write requested = %zu bytes, writing = %zu bytes\n",
221            length, valid_len);
222
223     /*
224      * Loop through each byte to be written:
225      * - Copy data from user space to kernel variable using get_user()
226      * - Write that byte to the memory-mapped peripheral using iowrite32()
227      */
228     for (i = 0; i < valid_len; i++) {
229
230         char data;
231
232         /*
233          * Copy one byte from user space buffer to kernel space.
234          * 'buffer' is a user-space pointer, so get_user is required.
235          */
236         get_user(data, buffer + i);
237
238         /*
239          * Write the byte to the corresponding offset in the
240          * multiplier peripheral address space.
241          */
242         iowrite32(data, virt_addr + i);

```

```

244         /* Increment count of successfully written bytes */
245         bytes_written++;
246
247         /* Debug print for each byte written */
248         printk(KERN_INFO "Multiplier: Wrote byte[%d] = %d\n", i, data);
249     }
250
251     /*
252      * Return the number of bytes successfully written to the device.
253      * This satisfies the requirement of the write() system call.
254      */
255     printk(KERN_INFO "Multiplier: Total bytes written = %d\n", bytes_written);
256
257     return bytes_written;
258 }
259
260 /* These define info that can be displayed by modinfo */
261 MODULE_LICENSE("GPL");
262 MODULE_AUTHOR("Nithin Gowtham Saravanan");
263 MODULE_DESCRIPTION("Module which creates a multiplier character device and allows user interaction with it");
264
265 /* Here we define which functions we want to use for initialization
266    and cleanup */
267 module_init(my_init);
268 module_exit(my_exit);
269

```

• multiplier.bb

```

6-character-device-driver > multiplier > E multiplier.bb
1  SUMMARY = "Recipe for build an external multiplier Linux kernel module"
2  SECTION = "PETALINUX/modules"
3  LICENSE = "GPLv2"
4  LIC_FILES_CHKSUM = "file://COPYING;md5=12f884d2ae1ff87c09e5b7ccc2c4ca7e"
5
6  inherit module
7
8  INHIBIT_PACKAGE_STRIP = "1"
9
10 SRC_URI = "file://Makefile \
11            file://multiplier.c \
12            file://COPYING \
13            file://xparameters.h \
14            file://xparameters_ps.h \
15            file://multiplier.h \
16            "
17
18 S = "${WORKDIR}"
19
20 # The inherit of module.bbclass will automatically name module packages with
21 # "kernel-module-" prefix as required by the oe-core build environment.

```